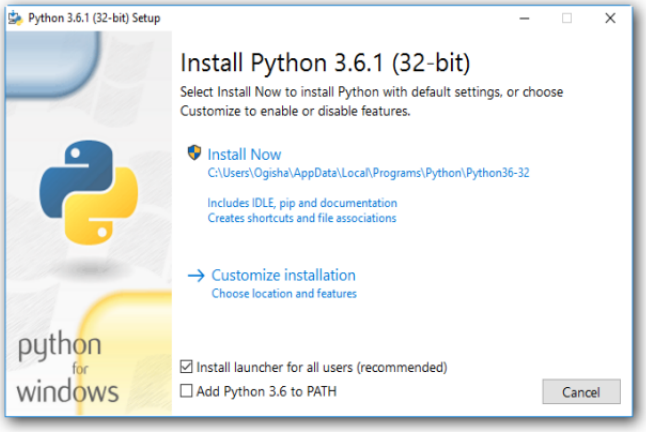
Основе програмирања и програмски језици

1 Вежбе 1: Основни типови података, операције, input/output

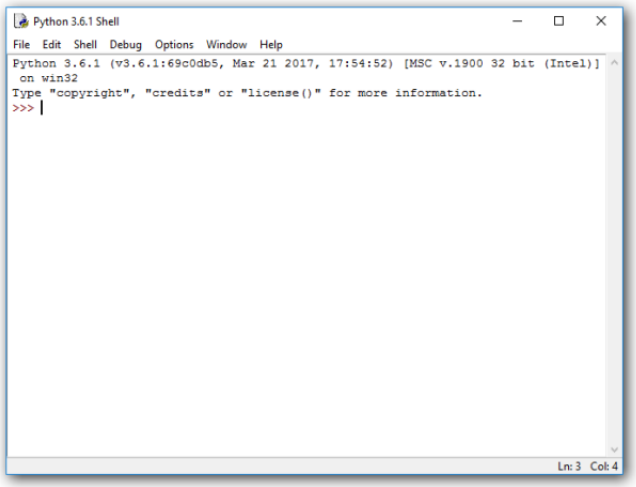
1.1 Инсталација

Са адресе `https://www.python.org/downloads/` преузети инсталациони фајл за одговарајућу верзију оперативног система који се користи. Покренути инсталацију као што је приказано на Слици 1.



Слика 1: Покретање инсталације Python-а

Уз основну инсталацију добија се бесплатно развојно окружење IDLE, које укључује итерактивни интерпретер, едитор текста и алате за тестирање кода.



Слика 2: Python IDLE

Симбол >>> означава текућу линију за унос команде која се, по притиску на тастер *Enter*, извршава од стране интерпретера, а њен резултат исписује у следећем реду. Провера да ли је Python инсталиран може се урадити командом **python - -version** у оквиру Command Line на Windows

систему или у оквиру терминала на Linux системима. Рад у интерактивном окружењу није погодан за извршавање сложенијег поступка јер се наредбе које га реализују, морају запамтити и поновити увек у истом редоследу. Зато се наредбе групишу у програм који је записан у датотеци на диску. Програм се може покренути, како из IDLE-а тако и ван њега. Екстенизја програма је **.py** . Покретање програма се обавља позивањем команде у виду **python moj_program.py**.

1.1.1 Едитори

Неке од најпопуларнијих Python едитора и развојних окружења (IDE), подељених према нивоу искуства и потребама пројекта су рецимо:

а) За почетнике:

- Mu Editor(`https://codewith.mu/`): Изузетно једноставан едитор направљен специјално за почетнике. Нема компликованих менија и идеалан је за учење основа.
- Thonny (`https://thonny.org/`) : Одличан је јер има уграђен визуелни дебагер који вам корак по корак показује како се променљиве мењају током извршавања кода.

б) Најпопуларнији

- Visual Studio Code (`https://code.visualstudio.com/`): Тренутно најпопуларнији бесплатни едитор на свету. Изузетно је брз, а уз званичну Python екстензију нуди врхунско ауто комплетирање кода, дебаговање и интеграцију са Git-ом.
- PyCharm (`https://www.jetbrains.com/pycharm/`): Комплетан IDE креиран специјално за Python. Долази у бесплатној (Community) и плаћеној (Professional) верзији. Бесплантна верзија је савршена за програмирање, док је плаћена намењена за напредни развој (Django, Flask).

в) Nauka, Nauka o podacima

- Jupyter Notebook (`https://jupyter.org/`): Ради директно у веб прегледачу. Уместо класичних фајлова, код пишете у ћелијама, што вам омогућава да одмах видите резултате, графиконе и табеле поред самог кода.
- Spyder (`https://www.spyder-ide.org/`): Долази у пакету са Anaconda дистрибуцијом. Изгледом подсећа на MATLAB и идеалан је за научнике и инжењере који се баве анализом података.

1.2 Основне аритметичке операције

Основне аритметичке операције су сабирање (+), одузимање (−), множење (∗), дељење (/), где важи стандардни приоритет операција. Промена приоритета операција, захтева стављање заграда. Додатно имамо и степеновање (∗∗), целобројно дељење (//) и остатак при дељењу (%). Примери основних операција су дати у Листингу 1.

Листинг 1: Основне операције

```
>>> 7 + 3
10
>>>7 - 4
3
>>>7 * 2
14
>>>7 / 2
3.5
>>>1 + 5 * 2
11
```

```
>>>7 + 8 / 4 - 5
4.0
>>>(1 + 5) * 2
12
>>>7 + 8 / (4 - 5)
-1.0
>>>5 ** 2
25
>>>127 // 10
12
>>> 127 % 10
7
```

Уколико направимо грешку, биће исписана одговарајућа порука. Погрешан део кода се аутоматски уклања и наставља се са радом (видети Листинг 2 и Листинг 3).

Листинг 2: Синтаксна грешка

```
>>> 5 *
SyntaxError: invalid syntax
>>> 5 * 2
10
```

Листинг 3: Семантичка грешка

```
>>> 20 / 0
Traceback (most recent call last):
 File "<pyshell#2>", line 1, in <module>
 20 / 0
ZeroDivisionError: division by zero
>>> 20 / 10
2.0
```

Задатак 1.

Шта ће се исписати ако унесемо следеће:

а) $50/5**2$

б) $10 + 25//7 - 3\%2$

в) $3 + 50\%2 * *3//2$

г) $3 + 50\%2 * *(3//2)$

Листинг 4: Решење Задатка 1.

```
>>> 50 / 5 ** 2
2.0
>>>10 + 25 // 7 - 3 % 2
12
>>>3 + 50 % 2 ** 3 // 2
4
>>>3 + 50 % 2 ** (3 // 2)
3
```

1.3 Коментари

Коментаре у оквиру кода користимо да појаснимо код, да учинимо код читљивим и за превенцију извршења при неком тестирању. Коментари почињу ознаком # и њих Python игнорише при извршавању (видети Листинг 5).

Листинг 5: Пример коментара

```
# Ovo je komentar. Ovaj kod stampa tekst
print("Pozdrav")

# Ovo je komentar. Ovaj kod stampa tekst
# Ovo je drugi komentar
print("Pozdrav")

print("Hello, World!") #Ovo je isto komentar

#print("Hello, World!")
print("Zdravo!")
```

Мултилинијски коментар се може додати убачен између ознака ” ” ” (видети Листинг 6). Наравно текст (string) у Python-у се дефинише преко знакова наводника али ће га Python игнорисати и третирати као коментар све док није додељен некој променљиви/величини.

Листинг 6: Пример вишелинијског коментара

```
"""
Ovo je komentar
napisan u
vise od jedne linije
"""
print("Hello, World!")
```

1.4 Променљиве

Променљиве се користе да складиште вредности и податке. Променљива се креира чим јој доделимо неку вредност (видети Листинг 7).

Листинг 7: Пример дефинисања променљиве

```
x = 5
y = "Milos"
print(x)
print(y)
```

Име променљиве сме садржати само бројеве, мала и велика слова, доњу цртицу (_) а не сме бити кључна реч и не сме почињати бројем. Python разликује велика и мала слова (видети Листинг 8).

Листинг 8: Разликовање великих и малих слова

```
>>> Duzina = 4
>>> Sirina = 5
>>> Povrsina = duzina * sirina
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'duzina' is not defined
```

У програмском језику Python , варијабле немају фиксне типове које морате унапред декларисати, већ тип варијабле зависи од вредности која јој је додељена (динамички типови података).

Кључне речи

Кључне речи у Python-у су: **and, or, not, True, False, None, if, elif, else, for, while, break, continue, match, case, def, class, lambda, return, yield, try, except, finally, raise, assert, with, import, from, as, global, nonlocal, del, pass, is, in, async, await.**

Главни и најцешће коришћени типови података у Python-у подељени су у неколико категорија: нумерички тип података-бројеви (int/integer - цели бројеви, float - децималне вредности, complex - комплексни бројеви), текстуални тип (str/string), логички тип (bool), колекције и низови (list - листа, tuple - торка, dict - речник, set - скуп), специјални тип (None - одсуство вредности или празна варијабла). Примери типова варијабли се могу видети у оквиру Листинга 9.

Листинг 9: Типови променљивих

```
x = "Hello World" #String, tekst
x = 20 #int
x = 20.5 #float
x = 1j #complex
x = ["apple", "banana", "cherry"] #list
x = ("apple", "banana", "cherry") #tuple
x = range(6) #range
x = {"name" : "John", "age" : 36} #dict
x = {"apple", "banana", "cherry"} #set
x = True #bool
x = None #NoneType
```

Провера типа променљиве се може урадити комадном **print(type(ime_promenljive))** . Директно дефинисање/промена типа променљиве се може урадити позивањем одређених функција (видети Листинг 10).

Листинг 10: Дефинисање/Промена типа променљивих

```
x = str("Hello World") #String, tekst
x = int(20) #int
x = float(20.5) #float
x = complex(1j) #complex
x = list(("apple", "banana", "cherry")) #list
x = tuple(("apple", "banana", "cherry")) #tuple
x =  dict(name="John", age=36) #dict
x = set(("apple", "banana", "cherry")) #set
x = bool(5) #bool
```

Python дозвољава дефинисање више променљивих у једној линији кода, дефинисање да више променљивих има исту вредност или извлачење/распакивање вредности из листи/торки (видети Листинг 11).

Листинг 11: Дефинисање више променљивих

```
#Vise promenljivih sa razlicitim vrednostima
x, y, z = "Orange", "Banana", "Cherry"
print(x)
print(y)
print(z)

#Vise promenljivih sa istom vrednoscu
x = y = z = "Orange"
print(x)
print(y)
print(z)
```

```
12
13 #Raspakivanje liste
14 fruits = ["apple", "banana", "cherry"]
15 x, y, z = fruits
16 print(x)
17 print(y)
18 print(z)
```

1.5 Испис/Output

Функција print() се најчешће користи за испис/output резултата/промељивих (видети Листинг 12).

Листинг 12: Испис променљивих

```
1  x = "Python je najbolji"
2  print(x)
3
4  #ispis vise promenljivih
5  x = "Python"
6  y = "is"
7  z = "najbolji"
8  print(x, y, z)
9
10 #ispis vise promenljivih, spojenog teksta
11 x = "Python "
12 y = "je "
13 z = "najbolji"
14 print(x + y + z)
15
16 #Operator + sabira ove dve promenljive - broja
17 x = 5
18 y = 10
19 print(x + y)
20
21 #ispis promenljivih razlicitog tipa
22 x = 5
23 y = "John"
24 print(x, y)
25
26 #viselinijski ispis jednom komandom print
27 print("""Kako
28 ste
29 danas?""")
```

Задатак 2.

Милош је на пијаци првог дана купило 10 јабука, другог дана 3, а трећег 8 јабука. Колико је Милош у просеку куповао јабука?

Листинг 13: Решење Задатка 2.

```
1 prvi = 10
2 drugi = 3
3 treci = 8
4
5 prosek = (prvi+drugi+treci)/3
6 print(prosek)
```

Задатак 3.

У корпи има 20 јабука. Потребно је да шесторо деце добије једнак број јабука. Колико јабука ће добити свако дете и колико јабука ће остати у корпи?

Листинг 14: Решење Задатка 3.

```
korpa = 20

dete = korpa // 6
print(dete)

ostatak = korpa % 6
print(ostatak)
```

Задатак 4.

Милош има 10 кликера. Петар има 2 пута више кликера од Милоша, а Никола има 5 кликера више од Милоша. Колико укупно кликера имају Петар и Никола?

Листинг 15: Решење Задатка 4.

```
milos = 10
petar = 2 * milos
nikola = 5 + milos
print(petar+nikola)
```

Задатак 5.

Израчунати површину правоуглог троугла, ако су дужине катета 5 и 3.

Листинг 16: Решење Задатка 5.

```
a = 5
b = 3
povrsina = 0.5 * a * b

print(povrsina)
```

1.6 Унос/Input

Уписивање текстуалне или бројчане вредности је могуће помоћу наредбе/функције **input()** . Сама наредба ништа не значи уколико није придружена некој променљивој. Тип прихваћених података је увек текстуални, уколико се не захтева другачије (видети Листинг 17). Измена типа се може урадити на начин као што је дато у Листингу 10.

Листинг 17: Унос променљивих

```
>>> a =  input("Uneti vrednost broja a: ")
12
>>> print(a)
12
>>> print(type(a))
<class 'str'>
```

Задатак 6.

Унети два цела броја x и y. На стандардном излазу одштампати вредност целобројног дељења броја x бројем y и вредност остатка при том дељењу.

Листинг 18: Решење Задатка 6.

```
x=int(input("Uneti broj x:"))
y=int(input("Uneti broj y:"))
a = x // y
b = x % y
print(a)
print(b)
```

Задатак 7.

Написати програм који за унети троцифрени цео број исписује збир цифара тог броја.

Листинг 19: Решење Задатка 7.

```
n=int(input("Uneti ceo trocifren broj: "))
a = n % 10 #Operacija ostatka pri deljenju sa 10 uvek "odseca" i vraca poslednju cifru broja (cifru jedinica).
n = n // 10  #Celobrojno deljenje sa 10 uklanja poslednju cifru i skracuje broj (npr. od 123 pravi 12). Ponavljanjem ovog postupka izolujemo svaku cifru pojedinacno, sleva nadesno.
b = n % 10
n = n // 10
c = n % 10
suma = a + b + c
print(suma)
```

Задатак 8.

Учитати две вредности са тастатуре и именовати их уз помоћ промењивих a и b. Размени вредности на које промењиве указују и потом их приказати на екрану.

Листинг 20: Решење Задатка 8.

```
a = input("Prva vrednost: ")
b = input("Druga vrednost: ")
print("Pre razmene a= ",a,"b= ",b)
c=a
a=b
b=c
print("Posle razmene a= ",a,"b= ",b)

#Drugi nachin razmene
a,b = b,a
```

Задатак 9 .

Шта ће бити вредност промењивих A и B након извршења наредби датих у Листингу 21?

Листинг 21: Решење Задатка 9.

```
A=4
B=7
A=2*A-B
B=7*A-B
A=A-A
print(A,B)
# A=0, B=0
```

1.7 Форматирање

У Python-у се ознака % користи за форматирање бројева унутар стрингова, што потиче из старијег стила форматирања (C-style % formatting). Списак свих главних ознака за форматирање:

- %d или %i - Цели бројеви
- %s - Текст
- %f - Децимални бројеви
- %.2f - Децимални број заокружен на тачно 2 децимале
- %.3f - Децимални број заокружен на тачно 3 децимале
- %e , %E - Експериментални/Научни запис бројева (Scientific notation)

Пример форматирања записа уз print() функцију је дат у Листингу 22.

Листинг 22: Форматирање исписа података

```
ime = "Marko"
godine = 25
visina = 1.87

# Primeri sa %d, %s i %f
print("Ime: %s, Godine: %d, Visina: %.2f m" % (ime, godine, visina))
# Izlaz: Ime: Marko, Godine: 25, Visina: 1.87 m
```

Иако је % стил исправан, у модерном Python-у се користе и друге методе (видети Листинг 23):

- f-стрингови
- .format() метода

Листинг 23: Други начини форматирања.

```
# f-stringovi
print(f"Ime: {ime}, Godine: {godine}, Visina: {visina:.2f} m")

#format metoda
print("Ime: {}, Godine: {}, Visina: {:.2f} m".format(ime, godine, visina))
```

1.8 Дијаграм тока

Термин дијаграм тока се односи на систем међусобно повезаних графичких симбола преко којих се представља алгоритам, односно, ток извршавања програма. Сваки појединачни симбол у дијаграму тока представља појединачни корак у алгоритму, а када је у питању редослед извршавања програмских инструкција, разликују се три основне опције:

- линеарно (безусловно) извршавање узастопних појединачних наредби

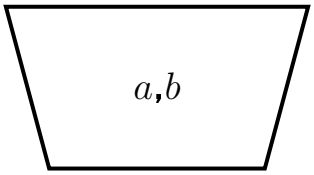
- гранање (прелазак на један од два могућа блока инструкција у следећем кораку - у складу са задатим условом)
- петља (узастопно/вишеструко понављање одређеног блока инструкција, такође сходно задатом услову)

На Слици 3 је приказан симбол који се користи у дијаграмима тока за почетак извршавања.



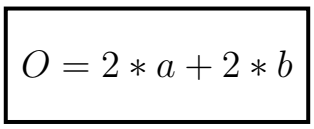
Слика 3: Симбол за почетак извршавања

Наредба за унос података, симболично се представља као трапез у коме се наводи један или више идентификатора променљивих које се учитавају (видети Слику 4).



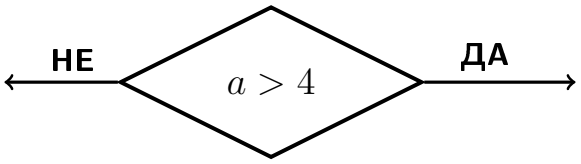
Слика 4: Симбол за наредбу улаза/уноса

У дијаграмима тока, правоугаоник означава појединачну наредбу за обраду података, а сама наредба се записује унутар правоугаоника (видети Слику 5).



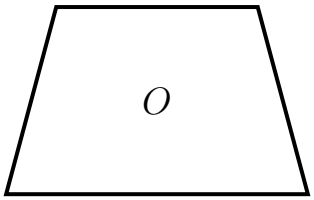
Слика 5: Појединачна наредба

Услови, који одређују даљи ток извршавања алгоритма, записују се унутар симбола у облику ромба, поред кога стоје ознаке ДА и НЕ (видети Слику 6).



Слика 6: Симбол за гранање у алгоритму

Наредбе за испис резултата се такође представљају преко симбола у облику трапеза, али, овога пута је доња основица већа (видети Слику 7).



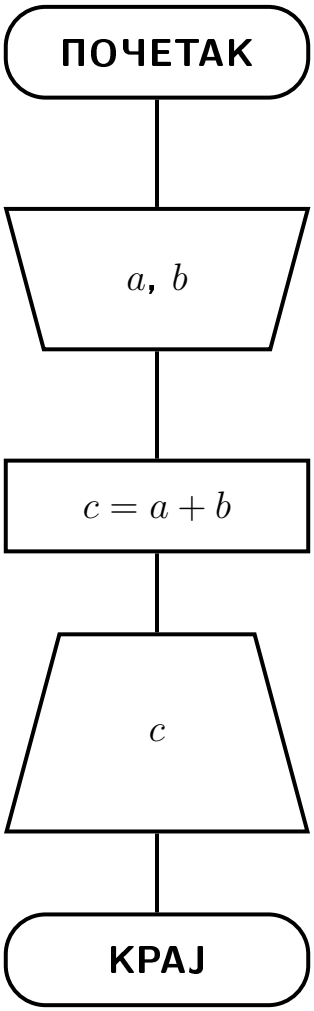
Слика 7: Симбол за наредбу за испис резултата

Симбол са Слике 8 представља завршетак извршавања.



Слика 8: Симбол за крај извршавања

На Слици 9 је приказан дијаграм тока који представља алгоритам за сабирање два броја.



Слика 9: Дијаграм тока који приказује линеарно/узастопно извршавање наредби

Додатне и сложеније примере дијаграма тока погледати на сајту датом у референци [3].

2 Самосталне вежбе 1

Задатак 1.

Река је дугачка 100 km и настаје од два тока. Први ток је за 32 km дужи од реке, а други ток је за 53 km краћи од реке. Израчунати укупну дужину све три реке.

Задатак 2.

Израчунати површину квадрата странице 5 cm, користећи оператор за степеновање. Добијену површину поделити са 2 и исписати посебно цео део броја, а посебно остатак.

Задатак 3.

Написати програм који за унети целобројни CIDR суфикс (нпр. корисник унесе `24` за `/24` мрежу) рачуна укупан број доступних хост адреса у тој подмрежи према формули:

$$N = 2^{32-CIDR} - 2$$

Користити оператор за степеновање. Резултат приказати као цео број (без децималних места).

Задатак 4.

Написати програм који двема унетим целобројним променљивим замењује вредности (другачији начин од онога датог у Листингу 20?).

Задатак 5.

Студент је желео да учита број година корисника, дода 5 година и испише резултат. Међутим, програм јавља грешку. Пронађите грешку и исправите код дат у Листингу 24.

Листинг 24: Код за Задатак 5.

```
godine = input("Koliko imate godina? ")
buduce_godine = godine + 5
print("Za 5 godina imacete:", buduce_godine)
```

Задатак 6.

Пронађите све грешке у коду датом у Листингу 25 који треба да израчуна и испише обим правоугаоника.

Листинг 25: Код за Задатак 6.

```
stranicaA = 10
stranicaB = 5
Obim = 2 * (stranicaa + stranicab)
print(obim)
```

Задатак 7.

Шта није у реду са кодом датим у Листингу 26?

Листинг 26: Код за Задатак 7.

```
cena = 100
poruka = "Cena artikla je: " + cena
print(poruka)
```

Задатак 8.

Написати програм који учитава реалан број x, а затим исписује прву децималу (прву цифру иза децималне тачке).

Задатак 9.

Написати програм који са улаза учитава троцифрени цео број, а затим формира нови тако што улазном броју замени прву и другу цифру гледано слева на десно. [пример: улаз:381, излаз:831]

Задатак 10.

Написати програм који од корисника тражи унос цене производа (реалан број) и проценат попуста (цео број од 0 до 100). Израчунати и исписати коначну цену производа заокружену на две децимале.

Задатак 11.

Корисник уноси број секунди (цео број). Написати програм који то време претвара у сате, минуте и преостале секунде. (Нпр. ако унесе 3665, програм треба да испише 1 сат, 1 минут и 5 секунди).

Задатак 12.

Написати програм који учитава четвороцифрен цео број, а затим рачуна и исписује производ прве и последње цифре тог броја.

3 Задаци за вежбање

Задатак 1.

Милош је дошао аутобусом на станицу у s_1 сати и m_1 минута, док је Иванов аутобус стигао у s_2 сати и m_2 минута. Колико је сати и минута онај који је први стигао чекао оног који је стигао други?

Задатак 2.

Група пријатеља је изашла на вечеру и укупан рачун износи R динара. На тај рачун се додаје бакшиш конобару у износу од 10% од укупног рачуна. Написати програм који учитава укупан рачун R и број пријатеља N, а затим рачуна и исписује колико динара треба да плати свака особа. Резултат исписати заокружен на две децимале.

Задатак 3.

Написати програм који помаже путнику да замени новац. Програм са тастатуре учитава износ у динарима (RSD) који путник има и тренутни курс евра (колико динара вреди један евро, нпр. 117.2). Програм треба да израчуна и испише колико ће евра путник добити, у формату: " За унети износ од [RSD] динара добијате [EUR] евра."

Задатак 4.

Написати програм који са тастатуре учитава дужину странице a једнакостраничног троугла (реалан број). Програм треба да израчуна и испише површину тог троугла. Корен рачунати преко оператора степеновања.

Задатак 5.

Корисник уноси четвороцифрен цео број (нпр. 4825). Написати програм који издваја и сабира средње две цифре тог броја (у примеру 4825, то су цифре 8 и 2, а њихов збир је 10). Исписати добијени збир на екрану.

Задатак 6.

Корисник уноси укупан број дана (нпр. 25 дана) као цео број. Написати програм који израчунава колико је то пуних недеља и колико преосталих дана (за 25 дана, резултат треба да буде: 3 недеље и 4 дана).

4 Литература/Корисни линкови

1. `https://imi.pmf.kg.ac.rs/moodle/course/view.php?id=327`
2. `https://peps.python.org/pep-0008/`
3. `https://codeblog.rs/clanci/dijagram_toka`

Датум ажурирања: 13.10.2026.